

X11 WM

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Introduction	2
2	Events	7
3	Tiling layout model	8
4	Layout recomputation	14
5	Focus model	15
6	Keyboard control	17
7	Mouse control	19
8	Borders and visual feedback	20

Introduction

- **The X11 WM role:** An X11 window manager is not the operating system's windowing system. It is a normal X client that is granted special control over the root window's child windows.

The X server owns the screen, keyboard, mouse, windows, pixmaps, cursors, and other graphical resources. Programs such as terminals, browsers, editors, and your window manager are X clients. They all talk to the X server through the X11 protocol.

A window manager's job is to manage other clients' top-level windows.

It usually controls:

- where top-level windows appear
- how large they are
- whether they are mapped or unmapped
- stacking order
- focus policy
- borders/decorations
- workspaces/tags/desktops
- fullscreen/maximized/minimized state
- how user input is interpreted as window-management commands

It does not normally draw the contents of applications. Firefox draws Firefox's content. Kitty draws Kitty's terminal grid. The window manager controls the container around those windows and their position in the global desktop.

Your window manager connects to the X server like any other client:

```
o Display* dpy = XOpenDisplay(NULL);
```

This gives your program a connection to the running X server.

The X server stores the actual window objects. In Xlib, a Window is just an integer ID referring to a server-side resource.

```
o Window root = DefaultRootWindow(dpy);
```

That root value is not a C object containing the window. It is an ID you send back to the X server when making requests.

Xlib calls such as **XMoveWindow**, **XResizeWindow**, **XMapWindow**, and **XDestroyWindow** send requests to the server. They do not directly manipulate local objects.

- **X Clients:** An X client is any program connected to the X server.

Each client may create one or more windows. Applications usually create a top-level window, which is the main window the window manager cares about.

A client creates a window using calls such as:

```
0  XCreateWindow(...)
1  XCreateSimpleWindow(...)
```

Then it usually maps the window:

```
0  XMapWindow(dpy, win);
```

Mapping means “make this window visible.” But because a window manager exists, the server may not immediately honor that mapping request. Instead, it may redirect the request to the window manager.

- **Top level windows:** A top-level window is usually an application window that is a direct child of the root window. These are the windows the window manager manages.

Not every window is top-level. Applications often create child windows inside themselves:

A basic window manager usually ignores internal child windows. It manages the top-level windows that live directly under the root window.

You can check a window's parent using

```
0  XQueryTree(...)
```

A window whose parent is the root window is usually a top-level client window.

- **Root window:** The root window is the invisible desktop-sized parent window for the whole screen.

Every X screen has a root window. It covers the entire display area.

For a simple one-screen setup:

```
0  int screen = DefaultScreen(dpy);
1  Window root = RootWindow(dpy, screen);
```

The root window is important because clients create their top-level windows as children of it.

The window manager selects events on the root window so it can intercept requests involving the root's child windows.

A minimal window manager usually does something like:

```
0  XSelectInput(  
1      dpy,  
2      root,  
3      SubstructureRedirectMask | SubstructureNotifyMask  
4  );
```

This is the critical step that makes your program the active window manager.

The window manager is “special” only because it selects `SubstructureRedirectMask` on the root window.

Only one client may select `SubstructureRedirectMask` on a given root window at a time.

That means only one window manager can run per screen.

If another window manager is already running, your attempt to select `SubstructureRedirectMask` fails with a `BadAccess` error.

Note: The X server creates the root window, not the window manager and not normal applications.

When Xorg starts, it initializes each screen. As part of that initialization, it creates one root window per screen. That root window is the top-level ancestor for all other windows on that screen.

- **SubstructureRedirectMask:** The `SubstructureRedirectMask` is an Xlib event mask that allows a single client (typically a window manager) to intercept structural requests made to the X server on the subwindows of a parent window

When this mask is set on a parent window (like the root window), it intercepts requests from other clients to map, configure (resize/move), or circulate child windows. Instead of executing the command, the X server pauses the action and sends the window manager a specific request even

- **SubstructureNotifyMask:** `SubstructureNotifyMask` is an Xlib event mask used primarily by window managers and system utilities. When selected on a parent window, it alerts the client to any structural or state changes affecting any of its direct child windows.

When the X Server receives `SubstructureNotifyMask`, it can report the following state-change events for children

- **CreateNotify:** A new child window was created.
- **DestroyNotify:** A child window was destroyed.
- **MapNotify:** A child window was mapped (made visible).
- **UnmapNotify:** A child window was unmapped (hidden).
- **ConfigureNotify:** A child window changed size, position, border, or stacking order.
- **ReparentNotify:** A child window was reparented to a different window.
- **CirculateNotify:** A child window's stacking order was circulated.
- **GravityNotify:** A child window was moved due to window gravity.

You should ideally install a temporary X error handler before this so you can detect whether another window manager already owns the root.

1. **normal client:** asks X server to map/configure its window
 2. **X server:** sees that WM selected `SubstructureRedirectMask` on root
 3. **X server:** sends event to WM instead of immediately performing request
 4. **WM:** decides what to do
 5. **WM:** sends `XMoveResizeWindow` / `XMapWindow` / `XConfigureWindow` / etc.
- **Reparenting vs non-reparenting window managers:** A window manager may either manage the application window directly or create a frame window around it.
 - **Non-reparenting WM:** A non-reparenting WM manages the application's top-level window directly.
 - **Reparenting WM:** A reparenting WM creates a frame window, then makes the client window a child of that frame.

```
root window
|__ frame window
    |__ client window
```

The frame belongs to the window manager. The client belongs to the application. This allows the WM to draw title bars, close buttons, resize handles, shadows, and borders independently of the client window.

- **Override-redirect windows:** Some windows are not supposed to be managed by the window manager. These are called **override-redirect** windows.

Your WM should usually ignore override-redirect windows.

- **Workspaces:** In a tiling WM, switching workspaces often means:
 - **Windows on current workspace:** Mapped
 - **Windows on hidden workspace:** Unmapped
- **Window lifecycle:** A top-level application window usually goes through this lifecycle:

1. client connects to X server
 2. client creates top-level window
 3. client sets properties/hints
 4. client asks to map the window
 5. WM receives MapRequest
 6. WM starts managing the window
 7. WM positions/resizes it
 8. WM maps it
 9. user interacts with it
 10. window may be resized/moved/focused/unfocused
 11. client may unmap it
 12. client may destroy it
 13. WM removes it from internal state
- **Events to understand:**
 - **MapRequest:** A client wants to show a top-level window.
 - **ConfigureRequest:** A client wants to move/resize/restack a top-level window.
 - **DestroyNotify:** A window was destroyed.
 - **UnmapNotify:** A window was hidden/unmapped.
 - **MapNotify:** A window was mapped.
 - **EnterNotify:** Pointer entered a window. Useful for focus-follows-mouse.
 - **ButtonPress:** Mouse button pressed. Useful for moving/resizing/focusing.
 - **KeyPress:** Key pressed. Useful for WM shortcuts.
 - **ClientMessage:** Client sent a protocol message, often EWMH/ICCCM-related.
 - **PropertyNotify:** A window property changed. Useful for titles, hints, states.
 - **Desktop backgrounds:** Setting the background of the desktop usually means setting the background of the root window.

So, we can convert an image into an X pixmap, set the roots background to that pixmap, and clear / redraw the root window.

Events

Tiling layout model

- **Intro:** A tiling layout model is the part of the window manager that decides: Given a monitor rectangle and a set of managed windows, what rectangle should each window occupy?

At minimum, a tiling window manager needs to track

```
0 struct Client {
1     Window win;
2     int x, y, w, h;
3
4     bool is_floating;
5     bool is_fullscreen;
6     bool is_hidden;
7
8     int min_w, min_h;
9     int max_w, max_h;
10    int base_w, base_h;
11    int resize_inc_w, resize_inc_h;
12
13    float min_aspect;
14    float max_aspect;
15 };
```

And something like:

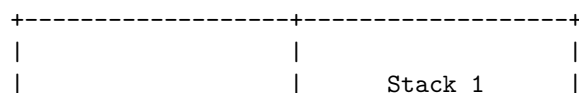
```

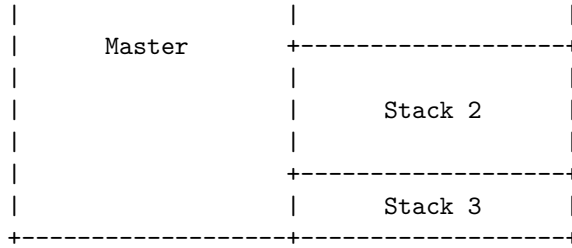
0 struct Workspace {
1     Client* clients[MAX_CLIENTS];
2     int client_count;
3
4     LayoutKind layout;
5
6     int master_count;
7     float master_factor;
8
9     int gap_inner;
10    int gap_outer;
11    int border_width;
12 };

```

The layout algorithm usually processes only tiled clients, floating and fullscreen clients are layered separately.

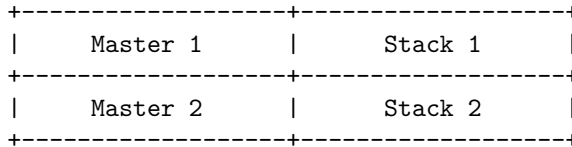
- **Master stack layout:** The screen is divided into two areas:



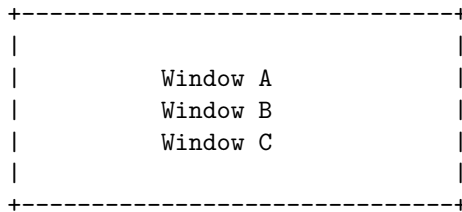


- The focused or first client goes in the master area.
- Remaining clients go in the stack area.
- `master_factor` controls how much width the master area receives.
- `master_count` controls how many windows are allowed in the master area.

With multiple master windows, the master area itself is split vertically.



- **Monocle / fullscreen layout:** Monocle means every tiled window receives the full usable monitor rectangle.



They are all the same size, but only the topmost/focused one is visible.

- **Monocle:** The WM still manages the window normally.

You may keep borders, gaps, and panels visible depending on your design.

- **Fullscreen:** Fullscreen is usually a special state.

A fullscreen client should typically:

cover the entire monitor, ignore gaps, ignore borders or use border width 0, appear above normal tiled clients, possibly cover bars/panels, be excluded from normal tiling.

In EWMH-aware WMs, fullscreen often corresponds to:

- **Grid layout:** Grid layout places windows in rows and columns.

A	B	C
D	E	F

For n clients, you choose

$$\begin{aligned} \text{cols} &= \lceil \sqrt{n} \rceil \\ \text{rows} &= \left\lceil \frac{n}{\text{cols}} \right\rceil. \end{aligned}$$

Then, for each cell,

```
cell_w = screen_width/cols
cell_h = screen_height/rows.
```

A common refinement is to avoid ugly empty cells. For example, with 5 windows:

A	B	C
D	E	

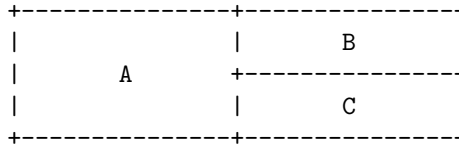
- **Binary space partitioning:** Binary space partitioning, or BSP, treats the screen as a recursively split rectangle.

The monitor begins as one rectangle:

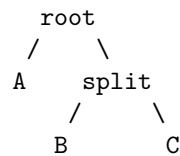
Diagram illustrating a root node structure. The root node is represented by a rectangle labeled "Root". The top and bottom edges are dashed lines, while the left and right edges are solid lines.

Then you split it

Then one side can split again:



Conceptually, you get a binary tree:



Each internal node is a split:

```

0  enum SplitKind {
1      SPLIT_HORIZONTAL,
2      SPLIT_VERTICAL
3  };
4
5  struct Node {
6      bool is_leaf;
7
8      union {
9          Client* client;
10
11         struct {
12             Node* first;
13             Node* second;
14             SplitKind split;
15             float ratio;
16         };
17     };
18 };
  
```

- **Manual split trees:** Manual split trees are BSP where the user controls where new windows go. For example,
 - Mod+h split horizontally
 - Mod+v split vertically

Then the next window is inserted according to that chosen split.

- **Dynamic tiling:** Dynamic tiling means the layout is generated automatically from the client list and layout mode.

In a dynamic tiler, the WM does not store a detailed layout tree for every split. Instead, it stores a list of clients and recomputes positions.

```

0  clients = [A, B, C, D]
1  layout = MASTER_STACK
2  master_factor = 0.6

```

Every time something changes, you call

```

0  arrange(workspace);

```

Events that trigger rearrangement include:

- new window mapped,
- window closed,
- focus changed,
- layout changed,
- master factor changed,
- monitor resized,
- gap size changed,
- client moved to/from floating state.

Dynamic tiling is simpler than manual tiling.

- **Floating layer:** Not every window should tile.

Some windows should float:

- dialog boxes,
- file pickers,
- small utility windows,
- splash windows,
- transient windows,
- user-forced floating windows,
- windows with fixed size hints.

The important design point is that floating windows are managed by the WM but excluded from tiling.

- **Gaps:** Gaps are empty space between windows

There are usually two kinds

- **Outer gaps:** Space between the monitor edge and the window layout
- **Inner gaps:** Space between adjacent windows

- **Borders:** Border width is part of the windows outer size. If you call

```

0  XMoveResizeWindow(display, win, x, y, w, h);

```

the w and h refer to the inner window size, not including the border.

- **Resize hints:** Resize hints come from the client application. In X11, you get them with:

```
0  XGetWMNormalHints
```

They are stored in **XSizeHints**. Important fields include

```
0  PMinSize      minimum width/height
1  PMaxSize      maximum width/height
2  PBaseSize     base width/height
3  PResizeInc    resize increment
4  PAspect       aspect ratio constraints
```

Some applications do not want arbitrary sizes.

- **Aspect-ratio constraints:** Aspect-ratio hints tell the WM that a window wants to stay within a certain width/height ratio.

For example,

```
0  min aspect = 16/10
1  max aspect = 16/9
```

The aspect ratio is

$$\text{ratio} = \text{width}/\text{height}.$$

If the width is too large for the height, reduce width.

If the height is too large for the width, reduce height.

Layout recomputation

- **Intro:** A tiling window manager should treat the screen layout as a derived result of its internal state.

That is the main idea behind layout recomputation. You do **not** want the geometry of each window to be the primary thing you manually maintain. Instead, you keep a clean model of the current world, then regenerate window positions from that model whenever something important changes.

state -> layout algorithm -> window rectangles -> XMoveResizeWindow calls

The window positions are an output, they are not the real source of truth.

- **Why you recompute the whole layout:** Suppose you have three windows A, B, C , and window B closes. If you only manually remove B , you now need to decide what happens to C .

The better model is

- **Before::** clients = [A, B, C]
- **After::** clients = [A, C]

Then rerun the layout algorithm.

- **New window appears:** When a new window appears, you usually receive something like MapRequest. You should
 1. decide whether to manage the window,
 2. add it to the current workspace's client list,
 3. set focus if appropriate,
 4. recompute the layout,
 5. map the window.
- **Window close:** When a window closes, you remove it from your model. Do not just “erase the dead window from the screen.” The X server already destroys/unmaps it. Your job is to update your internal state and then recompute the layout for the remaining windows.

Focus model

- **Intro:** A tiling window manager needs a focus model because X11 does not automatically know which client should receive keyboard input. Your WM must decide which window is “active,” tell the X server where keyboard events should go, and keep that decision consistent when windows are created, destroyed, hidden, moved between workspaces, or moved between monitors.

In X11 terms, focus mostly means keyboard focus. Pointer/mouse events are separate. A window can receive mouse events because the pointer is over it, while a completely different window receives keyboard events because it has input focus.

- **Input focus:** Input focus is the X11 mechanism that determines which window receives keyboard events.

The core call is usually

```
o XSetInputFocus(display, window, RevertToPointerRoot,  
  ↪ CurrentTime);
```

The focused window then receives keyboard input, assuming it selected the relevant events or has descendants that do.

A window manager usually does not focus the frame window for typing. It normally focuses the actual client window.

You may decorate/reparent the client into a frame, but keyboard focus should usually go to the client, not the decoration frame.

Focus is not just an X server state. Your WM also needs its own internal state saying which client is focused.

- **Click-to-focus:** Click-to-focus means a window becomes focused only when the user clicks it.

This is the easiest policy to implement and reason about.

- **Focus-follows-mouse:** Focus-follows-mouse means the window under the pointer becomes focused when the pointer enters it. The usual X event is **EnterNotify**.

you should usually ignore some EnterNotify events generated by grabs, ungrabs, or window hierarchy changes. X11 crossing events have fields like mode and detail that help distinguish normal pointer motion from synthetic/side-effect transitions.

You do not want accidental focus changes caused by:

- Mapping a new window
- Unmapping a window
- Moving/resizing a window
- Grabbing/ungrabbing pointer

- Switching workspace
- Restacking windows

So a real implementation often filters enter events.

- **Sloppy focus:** Variant of FFM. The difference is what happens when the pointer leaves a window and enters empty root-window space.
 - **Pointer over window A:** A focused
 - **Pointer over root:** A remains focused

Sloppy focus is often better for tiling WMs because there may be gaps, bars, or empty areas. You usually do not want focus to disappear just because the pointer moved over the root window.

- **Focus direction in a tiling layout:** A tiling WM often supports commands like:
 - focus left
 - focus right
 - focus up
 - focus down

There are two common ways to implement directional focus.

- **Layout-tree focus:** If your layout is stored as a tree, you navigate the tree
- **Geometry-based focus:** You compare window rectangles. If focused client is *A*, and the user says focus right, find the visible client whose rectangle is to the right of *A* and closest by distance.

Keyboard control

- **Intro:** In a tiling WM, keyboard control is not “text input.” It is a global command interface. You grab certain key combinations on the root window, receive KeyPress events before normal clients do, translate those events into internal commands, then mutate your WM state: focus, layout, workspace, client order, spawning, quitting, and so on.

```
physical key press
-> X11 KeyCode + modifier mask
-> WM matches key binding
-> dispatches command
-> command mutates WM state
-> WM rearranges / focuses / spawns / exits
```

- **Passive grabs:** A passive key grab means: “when this key combination is pressed, send the event to my WM.”

In Xlib, this is usually done with **XGrabKey**.

XGrabKey establishes a passive keyboard grab. When the matching key and modifier combination is pressed, the grab becomes active and the key event is delivered to the grabbing client, usually your WM.

For a WM, you usually grab keys on the root window, because the root window is the parent of all top-level client windows. This gives you “global” WM keybindings.

- **Modifier keys:** Modifiers are bitmask attached to key events

```
0  ShiftMask
1  LockMask      // usually Caps Lock
2  ControlMask
3  Mod1Mask      // often Alt
4  Mod2Mask      // often Num Lock
5  Mod3Mask
6  Mod4Mask      // often Super / Windows key
7  Mod5Mask
```

Most tiling WMs use Mod4Mask as the main WM modifier:

- **Key symbols:** A KeySym is the symbolic meaning of a key. For example,

```
0  XK_Return
1  XK_Tab
2  XK_space
3  XK_j
4  XK_k
5  XK_h
6  XK_l
7  XK_1
8  XK_q
```

These are layout-level symbolic names. You generally write your config using KeySyms because they are readable.

Internally you store keysyms, then when registering a grab you convert it to a keycode.

- **Key codes:** A KeyCode is the physical-ish code X11 reports for a key on the keyboard.

Your config should usually use KeySym.

Your grabs require KeyCode

- **Spawn commands:** A WM does not usually implement a terminal, launcher, browser, or editor itself. It spawns external programs.

A spawn command usually forks, closes the X connection in the child, starts a new session, then execs

Mouse control

Borders and visual feedback